

CODE ANALYZER AI

AI-Powered Code Explanation, Structural Analysis
and Complexity Estimation Platform

Project Documentation

Abstract

Code Analyzer AI is a web-based tool that helps a person understand a piece of code without reading it line by line. A user pastes a Python or JavaScript code snippet into the application, and the system returns four things: a plain English explanation of what the code does, a breakdown of the code's structure (such as the number of functions, loops, and conditions it contains), an estimate of how the code's running time and memory use will grow as the input grows, and a list of suggestions to improve the code.

The project is built using React for the user interface and Spring Boot (Java) for the backend logic. The backend uses a simple, rule-based parser to read the code and identify its building blocks, instead of using a heavy AST (Abstract Syntax Tree) parsing library. This keeps the system fast, lightweight, and easy to maintain. The frontend also keeps a short history of the last ten code snippets a user has analyzed. This history is stored locally in the browser, so the user can look back at previous results without needing an account or a database.

This project was built to address a problem that almost every developer faces at some point: spending too much time trying to understand code written by someone else, or code that they themselves wrote a long time ago and have since forgotten the details of. By giving a quick, structured summary of any code snippet, Code Analyzer AI saves time and lets the user focus on the more important parts of their work, such as fixing bugs or adding new features.

1. Introduction

Reading and understanding code is one of the most common tasks a software developer performs, often more common than writing new code from scratch. Developers regularly work with code written by other people, code from open-source projects, code inherited from a previous team member who has since left the company, or even their own code written months or years earlier. In all these situations, understanding what a piece of code does, how it is structured, and how efficient it is can take a significant amount of time, especially when the code has no comments or documentation.

Code Analyzer AI was built to make this process faster and easier. Instead of manually tracing through every line of a function, a developer can paste the code into the application and instantly receive a plain-English explanation of what the code does, along with a breakdown of its structure. The tool also estimates the time and space complexity of the code, which is useful when deciding whether a piece of code needs to be optimized. Finally, it offers suggestions on how the code could be improved, which is helpful both for learning purposes and for actual code review work.

The project is designed as a full-stack web application. The frontend, built with React, provides the user interface where a developer can write or paste code, select a programming language, and view the results. The backend, built with Spring Boot and Java, receives the code, processes it using a parsing engine, and returns the analysis as a structured response. The two parts of the system communicate using a REST API, which is a standard and widely used way for a frontend and backend to exchange data over HTTP.

This document describes the project in detail. It explains the problem the project solves, the objectives behind building it, the technology used, the system architecture, the design of both the frontend and backend, the algorithm used for analysis, the challenges faced during development, and the possible future improvements to the project.

2. Problem Statement

Software developers frequently encounter code that is difficult to understand quickly. This happens for several reasons. First, code written by someone else often does not follow the same style or logic that the reader is used to, which makes it harder to follow. Second, many codebases, especially older or legacy ones, have little to no documentation, leaving the reader to figure out the purpose of a function purely by reading its logic. Third, even well-written code can be time-consuming to analyze when a developer needs specific details, such as how many loops it contains, whether it uses recursion, or what its time complexity is.

Manually working out the time and space complexity of a function is also a skill that takes practice, and even experienced developers can spend a noticeable amount of time on it for anything beyond a simple example. Junior developers and students learning data structures and algorithms often struggle with this the most, since complexity analysis requires recognizing patterns such as nested loops, recursive calls, and how each one affects performance.

There is a clear need for a tool that can take a code snippet and quickly return useful information about it: what the code does, how it is structured, how efficient it is, and how it could be improved. Code Analyzer AI was built to solve exactly this problem.

3. Objectives

The main objectives of this project are as follows:

- To build a web application that can analyze Python and JavaScript code snippets and explain them in plain English.
- To detect and count structural elements in the code, such as functions, loops, conditional statements, and classes.
- To detect whether a piece of code uses recursion.
- To estimate the time complexity and space complexity of the submitted code.
- To generate simple, practical suggestions for improving the code.
- To keep a short history of recently analyzed code snippets, without requiring a database or user login.
- To design a clean and simple user interface that is easy to use for developers of all experience levels.
- To build the system using a lightweight parsing approach instead of a heavy AST library, keeping the project fast and easy to maintain.

4. Scope of the Project

Code Analyzer AI currently supports two programming languages: Python and JavaScript. A user can submit any function or short code snippet written in either language and receive an explanation, a structural breakdown, a complexity estimate, and optimization suggestions. The application is designed to work with individual functions or short snippets rather than entire codebases or multi-file projects, since the goal of the tool is to give a quick and focused analysis rather than a full project-level review.

The current version of the project does not include user accounts, authentication, or a database. All analysis history is stored locally in the user's browser, which means it is specific to the device and browser being used and is not shared across devices. This keeps the project simple and easy to set up, which is suitable for its current academic and demonstration purpose.

Support for additional languages such as Java, C++, C#, Go, and TypeScript, along with more advanced features such as AST-based parsing, user accounts, and cloud deployment, are planned as future enhancements and are discussed later in this document. They are outside the scope of the current implementation.

5. Technology Stack

The project is built as a full-stack application, with a clear separation between the frontend (what the user sees and interacts with) and the backend (where the actual analysis logic runs). The technologies used for each part are described below.

5.1 Frontend Technologies

- **React:** Used to build the user interface. React allows the application to be split into small, reusable components, such as the code editor, the results panels, and the history sidebar, which makes the codebase easier to manage and extend.
- **Axios:** Used to send HTTP requests from the frontend to the backend REST API and to receive the analysis results in return.
- **Monaco Editor:** The code editor component used in the application. It is the same editor that powers Visual Studio Code, and it provides syntax highlighting and a familiar coding experience for the user.
- **CSS:** Used for styling the application and making sure the interface looks clean and is easy to navigate.

5.2 Backend Technologies

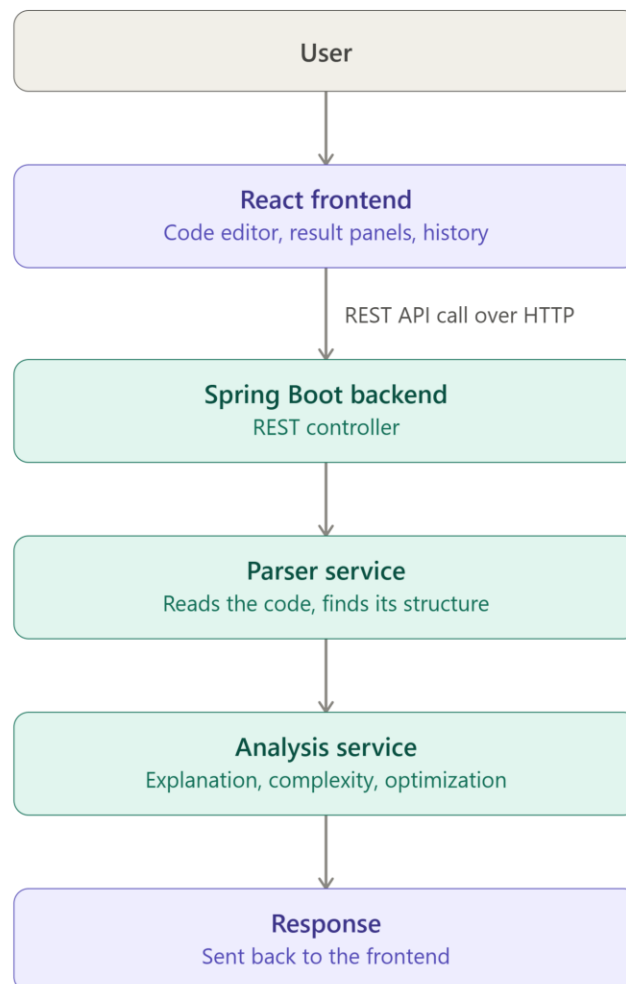
- **Spring Boot:** Used to build the REST API that the frontend communicates with. Spring Boot was chosen because it allows a Java backend to be set up quickly, with built-in support for handling HTTP requests, organizing code into layers, and managing configuration.
- **Java 17:** The programming language used to write the backend logic, including the parser, the complexity analyzer, and the explanation and optimization services.
- **Maven:** Used to manage the project's dependencies and to build and run the backend application.

5.3 Why This Stack Was Chosen

React and Spring Boot are two of the most widely used technologies in modern web development, which made them a practical choice for this project. They are well documented, have large communities, and are commonly required skills in the software industry, which made this project a useful way to practice and demonstrate real-world, full-stack development skills. Java was chosen for the backend specifically because of its strong support for structured, layered application design, which made it easier to separate the parsing logic, the analysis logic, and the API layer cleanly.

6. System Architecture

Code Analyzer AI follows a simple, layered architecture. The user interacts with the React frontend, which sends the submitted code to the Spring Boot backend through a REST API. The backend processes the code through a series of internal services and sends the combined result back to the frontend, which then displays it to the user. The overall flow is shown below.



This layered design keeps each part of the system responsible for one clear task. The frontend is only responsible for collecting input and displaying output. The backend's controller layer is only responsible for handling the incoming request and returning a response. The parser service is only responsible for reading the raw code and breaking it down into structural information. The analysis service then takes that structural information and produces the explanation, the complexity estimate, and the optimization suggestions. Keeping these responsibilities separate makes the system easier to test, debug, and extend in the future, for example when adding support for a new programming language.

7. Project Workflow

The following steps describe what happens, from start to finish, when a user analyzes a piece of code using the application.

1. The user opens the application in a web browser and is shown the home page, which includes a code editor and a language selector.
2. The user selects a programming language, either Python or JavaScript.
3. The user types or pastes a code snippet into the Monaco code editor.
4. The user clicks the “Analyze Code” button.
5. The frontend sends the code and the selected language to the backend using a POST request to the `/api/analyze` endpoint.
6. The backend's controller layer receives the request and passes the code to the Parser Service.
7. The Parser Service scans the code and identifies its structural elements, such as functions, loops, conditionals, classes, and recursive calls.
8. The structural information is passed to the Complexity Analyzer, the Explanation Service, and the Optimization Service, which each generate their part of the result.
9. The backend combines all of these results into a single response and sends it back to the frontend.
10. The frontend receives the response and displays the explanation, structure, complexity, and optimization suggestions in their respective panels.
11. The page automatically scrolls down so the user can see the results without needing to scroll manually.
12. The analyzed snippet is saved to the Recent Analysis History panel, unless an identical snippet already exists in the history.

8. Frontend Design

The frontend is organized into a set of components, each responsible for a specific part of the user interface. This makes the interface easier to build, test, and modify. The main components are described below.

8.1 Header

The Header component appears at the top of the page and displays the name of the application. It gives the user a consistent point of reference as they scroll through the page.

8.2 Code Editor

The Code Editor component wraps the Monaco Editor and provides the space where the user writes or pastes their code. It also includes the language selector, which lets the user choose between Python and JavaScript, and the “Analyze Code” button that triggers the analysis request.

8.3 Explanation Panel

The Explanation Panel displays the plain-English explanation of the submitted code, generated by the backend's Explanation Service.

8.4 Structure Panel

The Structure Panel displays the structural metrics returned by the backend, including the number of functions, loops, conditionals, and classes, along with whether the code is recursive.

8.5 Complexity Panel

The Complexity Panel displays the estimated time complexity and space complexity of the submitted code, presented in standard Big-O notation.

8.6 Optimization Panel

The Optimization Panel displays suggestions for improving the code, along with an optimized version of the code where one is available. This panel uses the React Diff Viewer library to show the original code and the optimized code side by side, making it easy for the user to see what changed.

8.7 History Sidebar

The History Sidebar displays the user's recent analyses, with the most recent one shown first. The user can click on a past entry to view it again, delete a single entry, or clear the entire history. This data is stored using the browser's local storage, so it

remains available even if the page is refreshed, but it does not carry over to a different browser or device.

9. Backend Design

The backend is built using Spring Boot and follows a standard layered structure, which is a common and widely recommended way to organize a Java web application. Each layer has a clear and separate responsibility, which is described below.

9.1 Controller Layer

The Controller Layer is responsible for handling incoming HTTP requests from the frontend. It receives the request at the `/api/analyze` endpoint, validates that the required fields are present, passes the code to the service layer for processing, and returns the final response to the frontend in JSON format.

9.2 Service Layer

The Service Layer contains the core business logic of the application. It coordinates the work of the Parser, the Complexity Analyzer, the Explanation Service, and the Optimization Service, and combines their individual results into a single response object that is sent back through the controller.

9.3 Parser Layer

The Parser Layer contains the logic that reads the raw code and identifies its structure. This includes detecting functions, loops, conditional statements, classes, and recursive calls. The parser used in this project is rule-based, meaning it looks for specific keywords and patterns in the code, rather than building a full syntax tree the way a compiler would.

9.4 DTO Layer

The DTO (Data Transfer Object) Layer defines the structure of the data sent between the frontend and the backend. This includes the request object, which holds the language and the code submitted by the user, and the response object, which holds the explanation, the structural metrics, the complexity estimate, and the optimized code. Keeping these definitions in a separate layer makes it clear exactly what data the API expects and returns.

10. Module Description

This section describes each module of the project in more detail, covering both the frontend and the backend.

10.1 Code Explanation Module

This module takes the structural information produced by the parser and turns it into a plain-English sentence or short paragraph describing what the code does. For example, if the parser detects a single function with one loop and one conditional that compares values inside the loop, the explanation module recognizes this as a common pattern (finding the largest or smallest value in a list) and generates an explanation accordingly.

10.2 Function Detection Module

This module scans the code for function definitions. In Python, it looks for lines that begin with the “def” keyword. In JavaScript, it looks for the “function” keyword, as well as arrow function patterns. Each match increases the function count returned in the structural analysis.

10.3 Loop Detection Module

This module scans the code for loop constructs, such as “for” and “while” in both Python and JavaScript. Care is taken to avoid counting these keywords when they appear inside a string (for example, inside a piece of text the program prints to the user), since this would otherwise lead to an incorrect loop count.

10.4 Conditional Detection Module

This module scans the code for conditional statements, such as “if”, “elif”, and “else” in Python, and “if” and “else” in JavaScript. The count of these statements gives a rough idea of how many decision points exist in the code.

10.5 Class Detection Module

This module checks whether the submitted code defines a class, using the “class” keyword in both Python and JavaScript. This is useful for understanding whether the code follows an object-oriented style or a purely functional one.

10.6 Recursion Detection Module

This module checks whether a function calls itself from within its own body. It does this by checking whether the name of a function appears again inside that same function's code, in a position where it is actually being called (rather than just appearing as a comment or inside a string). If a match is found, the code is marked as recursive.

10.7 Complexity Analyzer Module

This module estimates the time complexity and space complexity of the code based on the structural information gathered by the other modules. It looks at patterns such as the number of nested loops and whether the code is recursive, and matches these patterns to commonly known complexity classes, such as $O(1)$, $O(n)$, $O(n^2)$, and $O(\log n)$.

10.8 Optimization Module

This module looks for simple, well-known improvements that can be applied to the code, such as replacing a manual loop with a built-in function, removing unnecessary variables, or suggesting that a recursive function be rewritten as an iterative one when recursion is not strictly needed. It also generates an optimized version of the code where a clear, safe improvement is available.

10.9 Recent Analysis History Module (Frontend)

This module manages the list of recently analyzed snippets shown in the History Sidebar. It is responsible for saving a new snippet after analysis, checking for duplicates before saving, limiting the list to ten entries, removing the oldest entry when the limit is reached, and allowing the user to delete individual entries or clear the list entirely. All of this is handled using the browser's local storage, without any backend involvement.

11. Algorithm Used

The core of the backend is a rule-based parsing algorithm. Unlike a full compiler, which builds a complete syntax tree of the code, this algorithm works by scanning the code line by line and looking for specific keywords and patterns associated with each structural element. This approach was chosen because it is simple to implement, fast to run, and accurate enough for analyzing short code snippets, which is the main use case of this application.

11.1 General Steps of the Parsing Algorithm

13. Remove or ignore comments and string contents so that keywords appearing inside them are not mistakenly counted as real code.
14. Split the remaining code into individual lines.
15. Check each line against a set of patterns to detect function definitions, loops, conditionals, and class definitions, increasing the relevant counter each time a match is found.
16. Check whether any function name defined in the code appears again as a function call inside that same function's body, to detect recursion.
17. Pass the final counts and the recursion flag to the Complexity Analyzer and Explanation Service for further processing.

11.2 Example

Consider the following Python function, which finds the largest number in a list:

```
def find_max(numbers):  
    biggest = numbers[0]  
    for num in numbers:  
        if num > biggest:  
            biggest = num  
    return biggest
```

When this code is parsed, the algorithm detects one function definition (“def find_max”), one loop (“for num in numbers”), one conditional (“if num > biggest”), no class definitions, and no recursion, since the function does not call itself. These counts are then used to generate the structural analysis, the complexity estimate, and the plain-English explanation shown to the user.

12. Complexity Analysis Approach

Time complexity describes how the running time of a piece of code grows as the size of its input grows. Space complexity describes how the amount of memory the code uses grows as the input grows. Both are usually expressed using Big-O notation, which focuses on the general growth pattern rather than the exact number of operations.

Since Code Analyzer AI uses a rule-based parser rather than a full static analysis engine, its complexity estimates are based on recognizing common structural patterns rather than performing a complete mathematical analysis of the code. This approach gives a useful, practical estimate for the vast majority of typical functions, although it is not guaranteed to be perfectly accurate for every possible piece of code. The patterns used by the system are listed below.

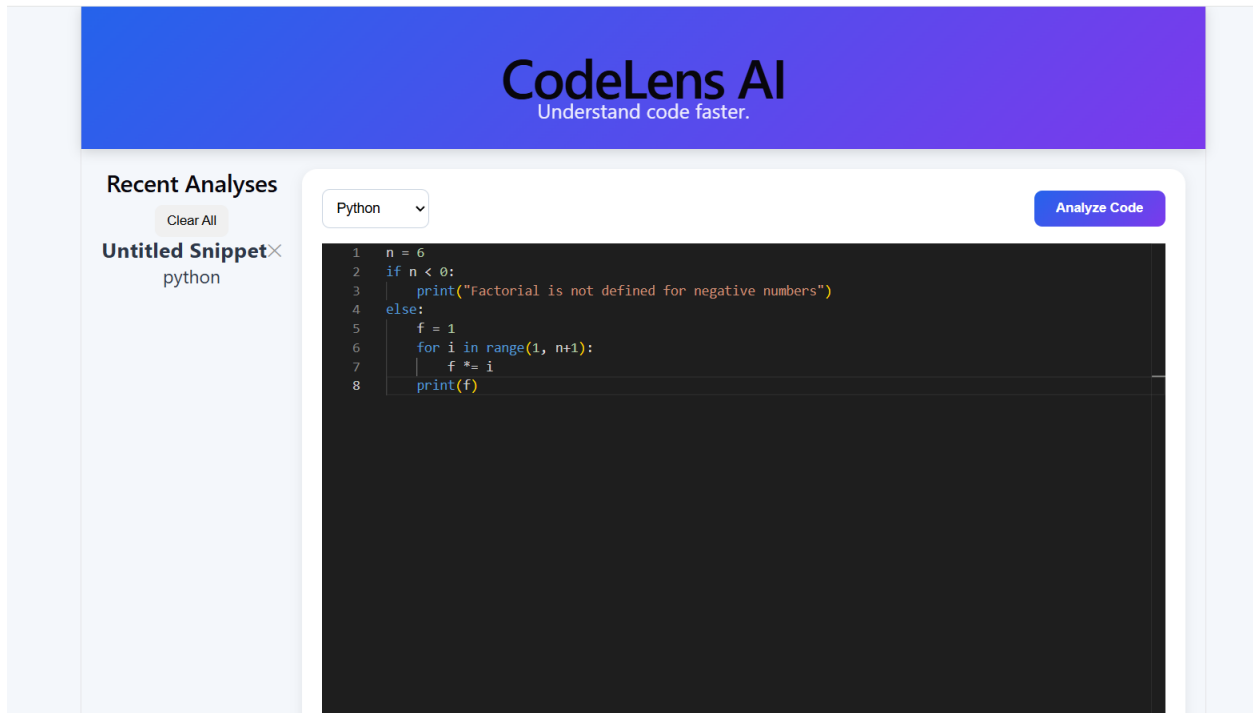
- Code with no loops and no recursion is generally estimated as $O(1)$, meaning its running time does not depend on the size of the input.
- Code with a single loop that runs through the input once is generally estimated as $O(n)$.
- Code with one loop nested inside another loop is generally estimated as $O(n^2)$.
- Recursive code where a function calls itself once per call, working through the input one step at a time, is generally estimated as $O(n)$ in time and $O(n)$ in space, due to the function calls building up on the call stack.
- Recursive code that splits the input in half on each call, such as a typical merge sort pattern, is generally estimated as $O(n \log n)$.

Space complexity is estimated in a similar way, by looking at whether the code creates new data structures that grow with the input size, and by accounting for the extra memory used by the call stack in recursive functions.

13. Screenshots

This section is reserved for screenshots of the application. Please replace each placeholder below with the corresponding screenshot before submitting this document.

Home Page



Analysis Results

Explanation

This Python code calculates the factorial of a given number `n`. It first checks if `n` is negative, and if so, prints an error message, otherwise, it uses a `for` loop to multiply all numbers from 1 to `n` and stores the result in `f`. The `range` function generates a sequence of numbers from 1 to `n`, and the `\*=` operator is used to multiply the current value of `f` with each number in the sequence. The final result is then printed to the console.

Code Structure

0

Functions

1

Loops

1

Conditionals

0

Classes

Complexcity Analysis of code



Optimized code and score

Confidence Score

92%

Optimized Code

```
```python
def calculate_factorial(n):
 if n < 0:
 return "Factorial is not defined for negative numbers"
 elif n == 0 or n == 1:
 return 1
 else:
 factorial = 1
 for i in range(1, n + 1):
 factorial *= i
 return factorial

n = 6
print(calculate_factorial(n))
```
```

14. Challenges Faced During Development

14.1 Detecting Recursion Accurately

One of the harder problems was correctly detecting when a function calls itself. A simple text search for the function's name was not enough, since the name could appear inside a comment, inside a string, or even as part of a different, similarly named function. The detection logic had to be made more careful to only count an actual function call as recursion.

14.2 Avoiding False Loop Detection

Since the parser works by searching for keywords such as “for” and “while”, it was initially possible for these words to be wrongly counted when they appeared inside a string, for example inside a printed message. This was fixed by making sure the parser ignores the contents of strings and comments before counting keywords.

14.3 Handling Duplicate Analysis History

When building the Recent Analysis History feature, it was important to avoid saving the exact same snippet more than once, since this would quickly fill up the list with repeated entries. A check was added to compare a new snippet against existing entries before saving it, so that duplicates are not added.

14.4 Designing a Lightweight Parser Without an AST Library

Choosing not to use a full AST parsing library meant that more care had to be taken when writing the parsing rules by hand, since there was no existing library to rely on for accuracy. This required testing the parser against a wide range of code examples to make sure the keyword-based approach produced reasonable results across different coding styles.

15. Future Enhancements

While the current version of Code Analyzer AI covers the core features described in this document, there are several improvements that could be added in the future to make the project more capable and production-ready.

- Integrating a large language model, such as one of OpenAI's models, to generate richer and more context-aware code explanations.
- Replacing the current rule-based parser with a full AST-based parser, to improve the accuracy of structural and complexity analysis.
- Adding support for more programming languages, including Java, C++, C#, Go, and TypeScript.
- Adding basic security analysis, to flag common unsafe coding patterns.
- Adding code smell detection, to highlight maintainability issues beyond just complexity.
- Deploying the application to a cloud platform, so it can be accessed without running it locally.
- Adding Docker support, to make the application easier to set up and run consistently across different machines.
- Adding the ability to export analysis results as a PDF document.
- Adding a user authentication system, so history can be saved to a user account instead of only the local browser.
- Adding support for team collaboration, so multiple users could share and discuss analysis results.

16. Conclusion

Code Analyzer AI demonstrates how a relatively simple, rule-based approach can still provide genuinely useful insights into a piece of code. By combining a React frontend with a Spring Boot backend, the project shows a complete, working example of a full-stack web application built around a clear and focused purpose: helping developers understand code faster.

Through this project, practical experience was gained in building a REST API, designing a layered backend architecture, writing a custom parsing engine, working with a modern frontend framework, and thinking through how to estimate algorithmic complexity in a structured way. The project also reflects an understanding of good software design principles, such as separating concerns between layers and keeping each module focused on a single responsibility.

While there is room for improvement, particularly around parsing accuracy and language support, the current version of Code Analyzer AI successfully achieves its core objective: turning an unfamiliar piece of code into a clear, structured, and easy-to-understand summary.

17. References

- React Documentation – official documentation for the React JavaScript library, used for building the frontend.
- Spring Boot Documentation – official documentation for the Spring Boot framework, used for building the backend REST API.
- Oracle Java Documentation – official documentation for the Java programming language and standard library.
- Monaco Editor Documentation – official documentation for the Monaco Editor component used in the code editor.
- Apache Maven Documentation – official documentation for the Maven build and dependency management tool.
- MDN Web Docs – reference material for JavaScript syntax and web development concepts.
- Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein – reference material for time and space complexity (Big-O notation) concepts.
- Axios Documentation – official documentation for the Axios HTTP client library used in the frontend.